Week 9 Lab 7: Power Management

Introduction

This lab is to introduce you to some ideas of power management, and some of the functionalities of your WeMOS. Compared to previous labs, this one is relatively short. So, after finishing, concentrate on your project!

In IoT, a lot of power management boils down to figuring out how to maximize the amount of time your device spends sleeping or idling. As we will see in the lecture this week, there are many functionalities built into microcontrollers and other ICs to allow us to minimize power consumption through power modes and gating.

Your **WeMOS** uses the microcontroller **ESP8266**, and this microcontroller is no different. It has four power modes:

- **Active mode:** The chip radio is powered on. The chip can receive, transmit, or listen.
- **Modem-sleep mode:** The CPU is operational. The Wi-Fi and radio are disabled.
- **Light-sleep mode:** The CPU and all peripherals are paused. Any wake-up events (MAC, host, RTC timer, or external interrupts) will wake up the chip.
- **Deep-sleep mode:** Only the RTC is operational and all other part of the chip are powered off.

We will now check out the power modes and test them ourselves.

Power Modes

First, let's look at the documentation of the ESP8266. It has been uploaded with this lab.

Task 1. What is the average power consumption of the WeMOS D1 when in Deep Sleep mode, modem-sleep, and when it is in active mode (you can assume it is only transmitting)? Assuming an ideal battery of 200mAh, how long would the device last in each state? **Hint:** look at the tech sheet. *(2 marks)*

Now, let's try out a Deep Sleep on the WeMOS. Create a new sketch and copy the following program:

```
void setup() {
  Serial.begin(9600);
  while(!Serial) { }
  Serial.println("Start device in normal mode!");
  delay(5000);
  Serial.println("I'm going into deep sleep mode for 20
seconds");
  ESP.deepSleep(20e6);  // Time in *microseconds*
}


void loop() {
}
```

Now upload this sketch to your WeMOS device. This example uses a timer to wake up your WeMOS device after putting it to deep sleep (all other part of the chips are powered off except the timer). After the time period is up, the timer will pull the pin **D0** high. Therefore, in order to wake up the WeMOS after the deep sleep, you will also need to connect **RST** to **D0** using one of your wires. You may need to reset your board with the button after uploading and wiring up.

Note that after the device wakes up from Deep Sleep, it is as if it has rebooted. It will run the setup routine again.

Important: you will not be able to upload the program with the wire connected to these two pins.

Task 2. Write a program (sketch) that reads a sensor value, connects to wifi, prints something to the serial, and then puts the WeMOS device into deep-sleep for 20 seconds. Include this program in your report. *(4 marks)*

For those further interested in extra modes, please take a look at the examples in **Examples > ESP8266 > LowPowerDemo**,

Other IC's also have their own power saving modes, Let's look at the power consumption of the **mpu9250** (the IMU device you have been given in your kit). I have included the tech sheet with this lab.

Task 3. What is the typical power consumption of the **mpu9250** when in normal mode for 9 axis measurements? What about in low power mode at 31.25 Hz? What do you think are the limitations of these modes? *(4 marks)*

Deadline is 5 days after your lab session. You only need to submit your report and any relevant code should be attached into it.